

Skill Function Description

Detailed feature introduction, business pain mapping, and use cases

What the package does

BeeTrust is a TypeScript release desk for a cross-border New Zealand honey export case [11]. It transforms one trade intent into a coordinated evidence workflow, tests the workflow against controlled faults, and returns a release recommendation for broker review. The package is designed for Commerce: the output is not a general information answer, but an auditable business-process decision with evidence, next actions, and explicit human-review boundaries.

Supported case: Release 1000 jars of Manuka honey from New Zealand to Australia for Australian retail review.

The commercial problem

An export release decision is often distributed across several hand-offs. Scientific evidence may be separated from custody records; destination requirements may be checked using an old rule snapshot; Customs teams may prepare the same documents repeatedly; and a clean happy path may not expose what happens when a quantity, destination, document, or rule version changes. Destination-specific export requirements and certificate evidence must be checked before release [1-3], while tariff classification and export-clearance data must be current and complete [4-6]. BeeTrust brings these decisions into one traceable loop.

Business pain	Operational consequence	BeeTrust response	Observable indicator
Evidence is fragmented across teams and systems.	Slow manual reconciliation, missing references, and weak auditability.	Five-gate evidence matrix with structured evidenceRefs and next actions.	Baseline evidence coverage: 100%.
Batch identity can change between farm, processor, warehouse, and carrier.	A wrong or altered batch can pass through a hand-off unnoticed.	SHA-256 chain with previousHash, hash, headHash, and tamperedEventIds.	Supported custody tamper fault: BLOCKED.
Market-access requirements are destination-specific and time-sensitive [1-3].	Stale or incomplete evidence creates rework, delay, or compliance risk.	Versioned MPI rules and explicit checks for required evidence and freshness.	Australia snapshot: MPI-AU-2026.09, 7/7 checks PASS.
Classification and paperwork consume repeat operator time [4-7].	Broker preparation starts from scattered data and assumptions.	HS candidates, landed-cost estimate, invoice, packing list, and TSW draft objects.	Three draft documents; landed-value estimate NZD 13,020 in baseline.
The happy path does not prove resilience.	A small change can create a late-stage incident or unsafe release.	Clone-preserving red-team replay and fail-closed evidence monitoring.	Six supported faults detected as BLOCKED.

How the business loop works

```
Trade intent
-> plan and dispatch
-> [scientific evidence || custody || MPI market access]
-> Customs preparation
-> controlled adversarial replay
-> evidence-monitor release gate
-> broker review, next actions, and retained evidence
```

Three root Skills start independently. Customs waits for MPI market access; adversarial testing waits for all operational outputs; evidence-monitor is the final decision point. Same-level work is concurrent, but results are committed in deterministic plan order. The local runtime adapter uses a LangGraph StateGraph execution model [10, 11]. Every lifecycle event uses SkillMessage/v1, so the workflow can be inspected as a runtime trace rather than treated as an opaque answer [11].

Package-level feature summary

Feature	What the user can do	Why it matters commercially
Natural-language intake	Submit one trade sentence and let orchestration-hub extract product, origin, destination, quantity, unit, and priority.	Reduces the setup burden for a release desk and creates one case identity for downstream work.
Parallel evidence checks	Run fingerprint, custody, and MPI checks as independent roots.	Shortens the dependency-aware path and makes bottlenecks visible.
Versioned knowledge	Use dated MPI and Customs snapshots mapped to source IDs, code paths, and self-tests.	Makes domain assumptions inspectable and refreshable instead of hiding them in prompts.
Draft document generation	Produce commercial invoice, packing list, and draft-only TSW objects from case data.	Reduces repeated preparation while keeping broker submission and approval explicit.
Red-team replay	Inject six supported faults into a cloned case and observe the blocked gate.	Tests operational resilience before a real case reaches a release decision.
Evidence-based release gate	Receive RELEASE, REVIEW, or BLOCKED with a score, gate reasons, missing evidence, and next actions.	Turns fragmented checks into a repeatable control point and preserves human accountability.

1. orchestration-hub: Trade intent and workflow coordination

Feature introduction. This is the incident commander of the package. It parses a natural-language export intent, creates a dependency-aware six-node DAG, dispatches the downstream Skills, handles retry budget, and emits the shared SkillMessage/v1 lifecycle record. It makes the order and dependency of work explicit before any release conclusion is produced.

Commercial pain addressed. Operators otherwise have to decide what to check first, remember which checks depend on each other, and reconcile status messages from separate tools. That creates coordination delay and makes it difficult to prove that a final release decision used all required steps.

Core features.

- Intent parsing for product, origin, destination, quantity, unit, and urgent versus normal priority.
- A fixed business DAG: three parallel roots, Customs after MPI, adversary after operational evidence, and monitor last.
- Concurrent same-level execution with deterministic plan-order commits for repeatable demos and tests.
- Two-attempt task budget with TASK_RETRY and TASK_FAILED lifecycle events for transient handler errors.
- Plan visualization through renderPlanLevels and runtime trace through renderRuntimeTimeline.

Interface area	Implementation contract
Input	raw: string trade intent; optional TradeCase and TaskPlan context for handlers.
Output	TradeIntent, TaskPlan, and ordered SkillMessage/v1 lifecycle events containing sender, receiver, status, attempt, payload, and evidenceRefs.
Normal use case	A broker enters the supported Australia export sentence. The system creates one case ID and starts fingerprint, custody, and MPI work without waiting for unrelated checks.
Exception use case	An empty intent, missing handler, or dependency cycle fails explicitly. A transient handler error creates a retry event; an exhausted retry budget creates TASK_FAILED instead of silently continuing.
Business value	Reduces coordination overhead, prevents skipped stages, and creates the execution trace needed for audit and performance measurement.

2. fingerprint-evidence: Scientific batch evidence comparison

Feature introduction. This Skill parses a laboratory CSV or report extract, normalizes known honey-marker aliases, compares the sample to a reference batch, and returns similarity, confidence, anomalies, and an evidence reference. The rule is deterministic and explainable: completeness and per-feature relative distance drive PASS, REVIEW, or BLOCKED.

Commercial pain addressed. A release team may receive a file with different marker names, missing values, or borderline measurements. Manual comparison is slow and can hide an incomplete sample behind a simple pass or fail label.

Core features.

- Accepts comma, semicolon, tab, or colon-separated feature rows and skips comments and header rows.
- Normalizes aliases such as MG, DHA, HMF, sugar profile, pollen DNA, moisture, conductivity, and delta13C.
- Calculates tolerance, relative distance, completeness, similarity, and confidence for the available reference features.
- Labels missing data and tolerance breaches as anomalies with a deterministic lab evidence reference.
- Discloses that prototype similarity scoring is triage support, not a laboratory certificate, UMF licence, or MPI assurance.

Interface area	Implementation contract
Input	FingerprintInput: batchId, sampleCsv, reference marker map, and referenceBatchId.
Output	FingerprintResult: status, marker features, similarity, confidence, anomalies, anomaliesDetected, evidenceRefs, and disclaimer.
Normal use case	The sample matches the reference vector. The Skill returns PASS, similarity 1, confidence 1, no anomalies, and a stable lab evidence reference for the release matrix.
Exception use case	Malformed CSV, duplicate or non-finite values, missing markers, or a large mismatch returns BLOCKED; a borderline match returns REVIEW so a human can request a better sample or accredited review.
Business value	Reduces manual scientific triage and stops incomplete evidence from being treated as a clean match.

3. custody-ledger: Tamper-evident chain of custody

Feature introduction. This Skill builds and verifies a SHA-256 hash chain for farm, processor, warehouse, and carrier events [9]. Each event records a previousHash and hash; the final headHash represents the reproducible chain tip. The implementation verifies payload, link, ordering, and batch identity.

Commercial pain addressed. A cross-border batch passes through multiple physical and organizational hand-offs. If an event is edited or linked to a different batch, an ordinary list of events may not make the change obvious to the reviewer.

Core features.

- Starts from a GENESIS value and links each canonical event to the previous hash.
- Checks event IDs, timestamps, actors, locations, actions, quantities, batch IDs, and optional metadata.
- Returns the complete chained events, headHash, tamperedEventIds, and custody evidence reference.
- Provides verifyHashChain for independent verification of an already chained list.
- States the boundary clearly: this is an evidence-integrity layer, not a public blockchain or legal title registry.

Interface area	Implementation contract
Input	Ordered CustodyEventInput[] plus a batch ID when using runCustodySkill.
Output	CustodyResult with PASS or BLOCKED, chained events, previousHash, hash, headHash, tamperedEventIds, and evidenceRefs.
Normal use case	Four fixture events move the batch from farm harvest to processor pack, warehouse release, and carrier booking. The headHash is retained as evidence for the release gate.
Exception use case	Empty events, a wrong batch ID, altered payload, invalid timestamp, duplicate event ID, or broken hash link blocks the case and identifies affected event IDs.
Business value	Reduces the cost of manual reconciliation and exposes batch identity risk before it becomes a broker or customer dispute.

4. mpi-market-access: Destination-specific market access evidence

Feature introduction. This Skill evaluates a versioned market-access snapshot for the destination. It checks listed beekeeper status, harvest declaration, RMP, OMAR, export certificate, trade certification, evidence freshness, and rule-version alignment, following the public export and certificate guidance used by the project [1-3]. The Australia fixture uses MPI-AU-2026.09 and returns check-level evidence references.

Commercial pain addressed. Export requirements differ by destination and change over time. Missing or stale documentation is often discovered late, when the shipment or broker packet is already being prepared.

Core features.

- Resolves destination aliases such as AU and Australia to a versioned market rule.
- Checks mandatory evidence separately so the reviewer can see exactly what is missing.
- Applies a prototype freshness window of 0 to 365 days and detects evidence rule-version conflicts.
- Uses fail-closed behavior for unknown markets, stale evidence, missing OMAR, missing documents, and version mismatch.
- Returns source URLs and a disclaimer that the Skill recommends; it does not issue an MPI assurance or certificate.

Interface area	Implementation contract
Input	destination: string and MpiEvidence containing the evidence flags, OMAR state, declaration date, freshness days, and optional ruleVersion.
Output	MpiResult with status, destination, ruleVersion, checks, missingEvidence, evidenceRefs, sourceUrls, and disclaimer.
Normal use case	The Australia evidence snapshot satisfies all seven checks, including freshness and rule alignment, so Customs can proceed with a known market context.
Exception use case	Unknown destination, negative or stale freshness, missing mandatory evidence, expired OMAR, or mismatched ruleVersion returns BLOCKED with reasons and references.
Business value	Reduces rework and late compliance surprises by making destination evidence a visible gate before document drafting.

5. customs-clearance: Classification, landed cost, and broker-ready drafts

Feature introduction. This Skill creates candidate HS classifications, calculates a transparent landed-value estimate from case-supplied rates, performs an optional live fetch of the public NZ Customs tariff page, and generates commercial invoice, packing list, and draft-only TSW objects [4-7]. It retains assumptions instead of presenting a candidate as a final legal classification.

Commercial pain addressed. Customs preparation repeatedly combines product lines, values, freight, insurance, rates, and source checks. Manual assembly increases the chance of inconsistent values and makes it difficult to distinguish a verified source from a case assumption.

Core features.

- Maps honey or Manuka product descriptions to candidate HS 0409.00 entries using the tariff source as the external confirmation point; non-honey products return VERIFY_REQUIRED [4].
- Calculates goods value, freight, insurance, duty, levy, export GST, and estimated landed value with rounded totals.
- In live mode, fetches the public tariff page, checks a content marker, and records HTTP status, content length, and SHA-256 hash [4, 9].
- Generates COMMERCIAL_INVOICE, PACKING_LIST, and TSW_DRAFT with shared case data and evidence references.
- Sets submission=DRAFT_ONLY and produces manual verification assumptions when source content is unavailable or classification evidence is weak.

Interface area	Implementation contract
Input	CustomsInput: seller, buyer, origin, destination, currency, invoice lines, freight, insurance, case-supplied rates, classificationEvidence, and optional liveSourceLookup.
Output	CustomsResult with HS candidates, totals, validationIssues, sourceUrls, assumptions, source snapshot data, evidenceRefs, and three document objects.
Normal use case	The baseline estimates NZD 13,020 landed value, returns the natural honey candidates, and creates invoice, packing list, and TSW draft objects for broker review.
Exception use case	Missing parties, invalid lines, negative or non-finite rates return BLOCKED. An unavailable live tariff source returns REVIEW with a manual verification action. No TSW submission occurs.
Business value	Reduces repetitive preparation and calculation errors while preserving the broker's authority to confirm classification, rates, and submission.

6. trade-risk-adversary: Controlled fault injection and stress testing

Feature introduction. This Skill replays six supported fault modes against a deep clone of the case: fingerprint mismatch, custody batch tampering, missing documents, quantity mismatch, destination change, and rule-version conflict. It observes the resulting finding and expected BLOCKED gate without mutating the baseline [11].

Commercial pain addressed. A demo or pilot that only shows the clean path cannot prove that a small data change will be detected. Real incidents often come from disagreement between fields that were valid earlier in the workflow.

Core features.

- Maintains an explicit ALL_FAULTS catalogue so test coverage is inspectable and repeatable.
- Deep-clones the TradeCase before mutation, preserving the original baseline for comparison.
- Returns a finding with fault, title, severity, observed behavior, and expectedGate=BLOCKED.
- Supports dry-run catalogue mode and active-fault mode for both demonstration and automated testing.
- Makes incident behavior visible in the dashboard and in the evidence-monitor decision.

Interface area	Implementation contract
Input	Complete TradeCase plus zero or more FaultType values from the supported catalogue.
Output	AdversaryResult with status, activeFaults, findings, optional mutatedCase, and red-team evidence reference.
Normal use case	The dashboard begins with a RELEASE baseline, then the operator selects one fault. The cloned case is re-evaluated and the affected gate becomes BLOCKED.
Exception use case	Unknown faults or fields that cannot be mutated are reported as controlled findings; the source case is not changed. Every active supported fault is expected to block.
Business value	Reduces operational risk by turning resilience from a claim into a repeatable pre-release test with observable evidence.

7. evidence-monitor: Evidence matrix and final release gate

Feature introduction. This Skill is the final auditor. It applies one evidence matrix across scientific fingerprint, chain of custody, MPI market access, Customs clearance, and adversarial stress. It emits RELEASE, REVIEW, or BLOCKED with a score, gate reasons, missing evidence, and next actions.

Commercial pain addressed. Without one final gate, teams may combine a successful scientific check with missing market evidence or an unresolved Customs issue. A score alone can also hide missing references. The monitor makes release precedence explicit.

Core features.

- Requires all five release gates and at least one evidence reference for a passing gate.
- Uses decision precedence BLOCKED over REVIEW over RELEASE; missing evidence is never silently ignored.
- Produces an evidenceMatrix keyed by gate, a score, missingEvidence, and actionable nextActions.
- Returns a broker-review recommendation rather than claiming independent shipment authority.
- Renders a concise audit summary for terminal, API, and dashboard use.

Interface area	Implementation contract
Input	TradeCase containing the outputs of the six operational roles, including status and evidence references.
Output	MonitorResult with decision, score, five gates, missingEvidence, nextActions, and evidenceMatrix.
Normal use case	All five baseline gates pass and contain references, so the monitor returns RELEASE 100/100 and instructs the team to release to broker review while retaining hashes, snapshots, and drafts.
Exception use case	A missing role output, blocked upstream gate, review state, or empty evidence reference produces BLOCKED or REVIEW with a concrete reason and next action.
Business value	Converts distributed checks into a repeatable control decision, reducing unsafe release risk and making the work auditable.

End-to-end use cases

Use case A: clean release for Australian retail review

A broker or exporter enters the supported trade intent. orchestration-hub creates the plan and starts three root checks. fingerprint-evidence confirms the sample against the reference profile; custody-ledger verifies four hand-offs; mpi-market-access validates the Australia snapshot. Customs then produces the candidate classification, estimate, and draft documents. The adversary runs a baseline dry run, and evidence-monitor returns RELEASE only after all five gates have evidence.

Observed output	Value	Commercial meaning
Final decision	RELEASE 100/100	The package recommends broker review with a complete evidence bundle.
Execution shape	3 parallel roots; 10 dependency edges; 4 critical-path stages	Coordination and bottlenecks are visible instead of hidden in manual hand-offs.
Evidence	100% coverage across 5 gates	Every passing gate has a reference that can be retained with the case.
Documents	COMMERCIAL_INVOICE, PACKING_LIST, TSW_DRAFT	The broker receives consistent drafts without implying submission.
Trace	12 SkillMessage/v1 messages	The runtime can be reconstructed from structured lifecycle events.

Use case B: live Customs source check

The operator runs `npm run demo:live` or passes `liveSourceLookup=true`. customs-clearance fetches the public NZ Customs tariff page [4], verifies a tariff/classification content marker, and records a SHA-256 snapshot hash [9]. If the page is unavailable or cannot be verified, the result is REVIEW and the operator receives a manual tariff-verification action. This prevents a source outage from being mistaken for a verified tariff decision.

Use case C: controlled incident response

The operator runs `npm run demo:incident` and selects a fault. trade-risk-adversary clones the baseline, applies the selected mutation, and runs the downstream checks. Examples include changing the first custody event's batch ID, removing MPI documents, increasing invoice quantity, changing the destination, or creating a rule-version conflict. The mutated case must become BLOCKED, while the original baseline remains RELEASE.

Use case D: invalid-input and human-review branch

An incomplete or borderline input does not disappear into a generic error. A malformed fingerprint becomes BLOCKED; an unknown market becomes BLOCKED; an invalid Customs line becomes BLOCKED; an unverifiable live source becomes REVIEW. These branches give the operator a concrete next action and are covered by the acceptance report.

Use case E: developer and workspace integration

```
cd apps/web
npm install
npm run build
npm test
npm run acceptance
npm run demo:live
npm run demo:incident
```

The compiled dashboard is written to `apps/web/dist/beetrust-dashboard.html`. The public TypeScript API is re-exported from `apps/web/src/public-api.ts`. Each Skill also includes `SKILL.md`, TypeScript logic, tests, a resource, and a runnable example.

How the package solves the commercial pain

Pain reduction mechanism	What changes in the operating process	Prototype evidence and pilot measure
Less coordination delay	The plan identifies dependencies and runs independent roots concurrently instead of serializing every check.	3 parallel roots and 4-stage critical path; measure median case triage time in pilot.
Less evidence rework	Each gate carries evidenceRefs and the final matrix lists missing evidence and next actions.	100% baseline coverage; measure evidence completeness and rework count.
Less batch-integrity risk	Hash links make payload or batch changes observable before release.	Six incident replays remain baseline-safe; measure supported fault detection $\geq 95\%$.
Less document preparation	Case data is reused across classification, cost calculation, invoice, packing list, and TSW draft objects.	3 generated draft objects; measure manual document minutes and correction count.
More transparent decisions	The output is RELEASE, REVIEW, or BLOCKED with reasons and human next actions, never a hidden confidence claim.	198 self-tests and all acceptance checks passed; measure reviewer trust and resolution time.

Measured prototype indicators and limits

The executed deterministic fixture reports 108 estimated manual minutes and 32 estimated orchestrated minutes, a 70% time-reduction proxy. It also reports 100% evidence coverage, 100% supported-fault detection, 3 parallel root agents, 4 critical-path stages, and 12 runtime messages. These values are transparent prototype calculations marked TARGET_FOR_PILOT in the TypeScript model. They are not historical savings or a claim about Comvita's actual operations.

The enterprise context is synthetic and uses Comvita's public investor page only [8]. Before production use, an authorised exporter and broker must refresh MPI and Customs sources [1-7], replace fixture data with authorised records, confirm laboratory evidence, validate classification and rates, and retain human approval. BeeTrust recommends release to broker review; it does not issue certificates, replace accredited laboratory judgment, or submit to TSW [6].

Implementation evidence

Relevant files: apps/web/src/workflow.ts, apps/web/src/skills/*/index.ts, apps/web/src/acceptance.ts, apps/web/src/kpis.ts, apps/web/src/public-api.ts, apps/web/skills/*/SKILL.md, apps/web/knowledge/, and apps/web/evidence-source-register.json [11]. Executed validation: npm run build passed; npm test passed 198 cases with zero failures; npm run acceptance passed; npm run demo:live returned HTTP 200 with a verified tariff marker; npm run demo:incident replayed all six supported faults as BLOCKED while retaining a RELEASE baseline [11].

References

APA 7th edition format. Bracketed numbers in the body correspond to the entries below. Web sources were accessed September 26, 2026; dynamic regulatory and tariff pages should be revalidated before production use.

- [1] New Zealand Ministry for Primary Industries. (n.d.-a). Steps to exporting honey and bee products. Retrieved September 26, 2026, from <https://www.mpi.govt.nz/export/food/honey-and-bee-products/steps-to-exporting>
- [2] New Zealand Ministry for Primary Industries. (n.d.-b). Honey and bee products: Requirements. Retrieved September 26, 2026, from <https://www.mpi.govt.nz/export/food/honey-and-bee-products/requirements>
- [3] New Zealand Ministry for Primary Industries. (n.d.-c). Animal product export certificates. Retrieved September 26, 2026, from <https://www.mpi.govt.nz/export/export-requirements/export-certification/animal-product-export-certificates>
- [4] New Zealand Customs Service. (n.d.-a). Tariff classifications and rates. Retrieved September 26, 2026, from <https://www.customs.govt.nz/business/tariffs/tariff-classifications-and-rates>
- [5] New Zealand Customs Service. (n.d.-b). Clear your exports. Retrieved September 26, 2026, from <https://www.customs.govt.nz/business/export/clear-your-exports>
- [6] New Zealand Customs Service. (n.d.-c). Getting started with Trade Single Window. Retrieved September 26, 2026, from <https://www.customs.govt.nz/business/trade-single-window-tsw/getting-started>
- [7] New Zealand Ministry of Foreign Affairs and Trade. (n.d.). Guide to using free trade agreements for goods exporters. Retrieved September 26, 2026, from <https://www.mfat.govt.nz/en/trade/how-we-help-exporters/guide-to-using-free-trade-agreements-for-goods-exporters>
- [8] Comvita Limited. (n.d.). Investor centre. Retrieved September 26, 2026, from <https://comvita.co.nz/pages/investor-centre>
- [9] National Institute of Standards and Technology. (2015). Secure Hash Standard (SHS) (FIPS PUB 180-4). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.180-4>
- [10] LangChain, Inc. (n.d.). LangGraph overview. Retrieved September 26, 2026, from <https://docs.langchain.com/oss/javascript/langgraph/overview>
- [11] BeeTrust project team. (2026). BeeTrust Honey Export Release Desk [Computer software and source code]. Unpublished project repository.